WEEK 2 · LESSON 11 OF 24 · HANDOUT

Releasing Safely: Shadow Mode, Canary and A/B Tests

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=GN5PJXajxKw — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyBay 2019: A/B Testing in Python — https://www.youtube.com/watch?v=QWeMYnlNtzY

🎓 **Go deeper:** EuroPython 2020: Can we deploy yet? — https://www.youtube.com/watch?v=Fgyk8P0IJsU

Your goal in this lesson: Release a new model with low risk using shadow mode, canary releases and A/B tests, decide with statistics instead of feelings, and choose between batch and real-time scoring.

Never switch everyone at once

A new model can pass every test and still surprise you with real traffic. Release in steps:

Strategy	What happens	Risk to users
Shadow mode	The new model scores every request, but only the old model's answer is used	None
Canary	A small share (say 5–10%) of requests use the new model	Small
A/B test	Two groups get different models; you compare outcomes	Controlled
Full rollout	Everyone uses the new model; the old one stays ready for rollback	Watched closely

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

Shadow mode

```
from sklearn.ensemble import GradientBoostingClassifier

challenger = GradientBoostingClassifier(max_depth=2, n_estimators=150,
random_state=0).fit(train[FEATURES], train[TARGET])
traffic = make_applicants(n=2000, seed=40)

old = model.predict_proba(traffic[FEATURES])[:, 1] >= 0.5          # the answer users get
new = challenger.predict_proba(traffic[FEATURES])[:, 1] >= 0.5     # logged only
print(f"agreement: {(old == new).mean():.1%}")
print(f"approval rate  old {old.mean():.1%}  new {new.mean():.1%}")
diff = traffic[old != new]
print(f"{len(diff)} applicants would get a different decision; review a sample by hand:")
print(diff[FEATURES].head(3).to_string(index=False))
```

Shadow mode shows **how different** the new model is on real requests, at zero risk. Look at the disagreements by hand: are the new decisions sensible?

Canary routing

Send a fixed share of applicants to the new model. A hash of the ID keeps each person on the same model every time they apply:

```
import hashlib

def use_new_model(applicant_id, share=0.10):
    bucket = int(hashlib.sha256(f"canary-v2:{applicant_id}".encode()).hexdigest(), 16) % 100
    return bucket < share * 100

routed = traffic["applicant_id"].map(use_new_model)
print(f"share on the new model: {routed.mean():.1%}")
print("same answer for the same person every time:", use_new_model(123) == use_new_model(123))
```

If canary metrics look bad (errors, strange approval rate), turn the share back to 0% — one setting.

A/B test: is the difference real?

The SACCO tests a new approval policy. In group A (old policy) 640 of 800 approved loans were repaid; in group B, 668 of 800. Is B really better, or is it luck?

```
from scipy.stats import norm

def two_proportion_test(success_a, n_a, success_b, n_b):
    pa, pb = success_a / n_a, success_b / n_b
    pooled = (success_a + success_b) / (n_a + n_b)
    se = np.sqrt(pooled * (1 - pooled) * (1 / n_a + 1 / n_b))
    z = (pb - pa) / se
    return pa, pb, z, 2 * (1 - norm.cdf(abs(z)))

pa, pb, z, p_value = two_proportion_test(640, 800, 668, 800)
print(f"A {pa:.1%}  B {pb:.1%}  difference {pb - pa:+.1%}  p-value {p_value:.3f}")

pa, pb, z, p_value = two_proportion_test(3200, 4000, 3340, 4000)
print(f"same rates, 5x more loans: p-value {p_value:.4f}")
```

With 800 loans per group the p-value is about **0.07** — not quite convincing at the usual 0.05 level. With five times more loans, the same difference gives a tiny p-value. **Decide the sample size before the test**, and do not stop early just because the numbers look good — that is how many A/B tests go wrong.

Batch or real-time?

```
import time

start = time.perf_counter()
for i in range(200):
    model.predict_proba(traffic[FEATURES].iloc[[i]])
one_by_one = (time.perf_counter() - start) / 200

start = time.perf_counter()
model.predict_proba(traffic[FEATURES])
batch = (time.perf_counter() - start) / len(traffic)
print(f"one at a time: {one_by_one * 1000:.2f} ms each | batch: {batch * 1000:.4f} ms each "
      f"({one_by_one / batch:.0f}x faster per applicant)")
```

	Real-time (API)	Batch
When	An applicant waits for an answer	Scoring many records on a schedule (every night)
Speed per item	Slower	Much faster
Example	Loan decision at the counter	Monthly risk review of all active loans

Key terms

- **Shadow mode** — the new model runs on real traffic but its answers are not used.
- **Canary release** — a small share of traffic gets the new version.
- **A/B test** — a controlled comparison of two versions on two groups.
- **p-value** — how surprising the result would be if there were no real difference.
- **Batch scoring** — scoring many records together on a schedule.

Remember this: Release new models in steps — shadow, canary, A/B, full rollout — decide with pre-planned statistics, and pick batch or real-time scoring by how fast an answer is needed.

Name: _____ Date: _____

Activity

Task 1

Change the canary share to 25% and check the real share.

Task 2

Find the smallest group size (try 1,000, 1,500, 2,000…) at which 80% vs 83.5% has a p-value below 0.05.

Task 3

List three things you would check by hand in the shadow-mode disagreements.

Task 4

Decide batch or real-time for: SMS reminders to late payers, a USSD loan request, an annual portfolio report.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. In shadow mode, whose answers do users get?
   A. Neither
   B. Both
   C. The new model's
   D. The old model's; the new model is only logged
2. Why route canary traffic with a hash of the applicant ID?
   A. The same person always gets the same model
   B. To hide the model
   C. It is faster
   D. Hashes are random every time
3. What did a p-value of about 0.07 with 800 loans per group mean?
   A. A is better
   B. B is certainly better
   C. Not quite convincing at the usual 0.05 level
   D. The test failed to run
4. What is a common mistake in A/B tests?
   A. Stopping early because the numbers look good
   B. Recording results
   C. Planning the sample size
   D. Using two groups
5. Which is a good use of batch scoring?
   A. A chat reply
   B. A login page
   C. A loan decision at the counter
   D. A nightly review of all active loans
6. What should stay ready during a full rollout?
   A. Nothing
   B. A second office
   C. A new dataset
   D. The old model, for rollback

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.